第 5 章練習解答　程序同步

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS05-E01　critical section 性質

1. 破壞的是 bounded waiting。若任何時刻最多一個執行流在 critical section，mutual exclusion 成立；若離場後總有 candidate 能被選中，progress 也成立；但特定候選者仍可被無限次插隊。
2. 不一定。系統可以永遠有某個執行流進入並離開 critical section，因此有 global progress；被跳過的 thread 沒有完成時間界線，這是 starvation 而不是 deadlock。
3. 指定公平規則與上界，例如 FIFO ticket：每個 entrant 先原子取得遞增 ticket，只有 queue head 可進入；若 Pi 前面最多有 n−1 個已請求者，且每個持有者離開，Pi 必須在最多 n−1 次他人進入後取得許可。

檢查。 活性與公平是不同檢查。

OS05-E02　有界緩衝區

1. producer 先 wait(empty)，因為 empty=0 代表沒有空槽；此等待必須在取得 mutex 前。
2. consumer 已持有 mutex 並取出項目後，先 signal(mutex)，再 signal(empty)。被喚醒的 producer 甦醒後才能重新競爭 mutex，避免兩個執行流同時改 buffer。
3. 若 producer 先取得 mutex 再因 empty=0 阻塞，consumer 必須先取得同一個 mutex 才能取出項目並 signal(empty)，但它永遠等不到 mutex。等待容量條件與等待資料結構互斥鎖交叉，形成 deadlock。

檢查。 檢查所有 wait 順序與持鎖阻塞路徑。

OS05-E03　條件變數

1. condition variable 不保存 signal。wait 可能虛假喚醒、被 broadcast 喚醒，或醒來後 predicate 已被另一個 thread 消耗；一次 `if` 不足以形成穩定的等待條件。
2. 使用 `while (!ready) wait(cv)`；每次從 wait 返回後都在同一 mutex 下重新讀 predicate。
3. notifier 先取得 protecting mutex，修改 ready，再 signal，最後釋放 mutex；waiter 在同一 mutex 下檢查 ready，若 false 則原子地釋放 mutex 並入 condition queue。若 signal 先發生，waiter 會看到 ready=true；若 waiter 先入 queue，signal 可找到它，因此不會 lost wakeup。

檢查。 condition variable 管喚醒，predicate 管狀態。

第 5 章練習解答　程序同步 （續）

OS05-E04　Peterson 正確性與記憶體順序

1. 固定atomic seq_cst讀寫，反設P0/P1各在本輪critical section且重疊。比較兩者本輪進入前最後的turn寫入：若P1的turn=0較晚，P0本輪的flag0=true必先於P0寫turn，因而先於P1此次寫turn；P0在離開本輪CS前不清flag0，且兩者在本輪CS內都不再寫turn。P1要在P0尚未離場時通過while，會看到flag0=true、turn=0，條件flag0&&turn==0為true而受阻，矛盾。若P0的turn=1較晚，對稱使P0受阻。故不存在重疊CS；不能把P1寫turn0說成使自己的turn!=0。
2. Pi 被擋只可能是 flag[j]=true 且 turn=j。Pj 在 critical section 內不改 turn；若 Pj 再次想進入，必須重寫 turn=i，因而會讓 Pi 取得優先。故 Pi 在有限時間內至多等 Pj 一次進出。
3. progress 分為單一、雙候選：單一候選看到對方 flag=false 可進；雙候選時 turn 必等於其中一方，因此有一方可進。mutual exclusion、progress 與 bounded waiting 同時成立，來源是 atomic seq_cst 讀寫與每個 thread 持續前進的假設。
4. 非 atomic 的 flag/turn 讀寫沒有跨 thread happens-before，編譯器或 CPU 可重排或延遲可見性；逐步 interleaving 證明不再涵蓋實際執行。必須改用相應原子操作、fence 或鎖，不能直接套用原證明。

檢查。 必須同時引用 turn 的 persistency、seq_cst 可見性與 bounded wait，不能只列名稱。

OS05-E05　Semaphore 狀態與 bounded buffer

1. 初始化 buffer=[]。put A：empty2、full1、buffer[A]；put B：empty1、full2、buffer[A,B]；put C：empty0、full3、buffer[A,B,C]；get A：empty1、full2、buffer[B,C]；put D：empty0、full3、buffer[B,C,D]。mutex 每次 critical section 後回到1。
2. 區分binary抽象permit0/1與此處signed blocking counter。S初值1，T0 wait先減1→0取得permit；T1 wait再減0→−1並入queue睡眠。T0 signal先加−1→0，因加後S<=0而喚醒一個waiter，permit直接handoff給T1；T1被喚醒後延續原wait，不再wait或重減。S=0此時沒有free permit，但已授與T1。T1稍後signal0→1時沒有waiter才留下free permit。一般signal規則是先S++，若S<=0則wake一個，不能只在0→1時喚醒。
3. 若 producer 持 mutex 並因 empty=0 阻塞，consumer 雖然 full>0，仍要等待同一 mutex；形成 producer 等 empty、consumer 等 mutex 的等待環。容量 wait 必須在 mutex 外，進入 mutex 後才能操作 slot。

第 5 章練習解答　程序同步 （續）

檢查。 上表列的是沒有在途 reservation 的完成邊界，此時 E+F=3。一般原子轉移邊界以非負未保留 permits E/F、producer 在途 P 與 consumer 在途 C 檢查 E+F+P+C=3；成功 wait 即取得 reservation，直接 handoff 也算授出。初始 (3,0,0,0) 在一次 wait(empty) 後為 (2,0,1,0)，故中途 E+F=2。signed semaphore raw 值含 queued waiters，不可當 E/F；occupancy 的安全條件另為 0<=items<=3，且發布必須在對應資料更新之後。

OS05-E06　Condition variable、Mesa 與 lost wakeup

1. 若先釋放 monitor 再加入 queue，另一 thread 可剛好在兩步間 signal；此時 queue 仍空，signal 消失。wait 的原子契約保證 release-lock 與 enqueue 不是可分開的交錯。
2. 若 signal 先發生，通知者已在同一 mutex 內令 ready=true。waiter 之後取得 mutex 先檢查 ready，發現 true，根本不呼叫 wait。若 waiter 先開始，它在同 mutex 下檢查後原子入 queue，signal 才能找到它。
3. Mesa 的 signal 只把 waiter 移到 ready queue，通知者仍持有 monitor；waiter 重新取得 monitor 後 predicate 可能已被其他 thread 改回 false。因此必須再以 while 檢查，不能假設 signal 是 permit。
4. Hoare signal-and-wait 立即轉交 monitor，predicate 在交接時保持真，常用 if；成本是 context switch 與 signaler 被延後。Mesa signal-and-continue 保留 signaller，waiter 必須重取鎖並在 while 檢查 predicate；POSIX condition variable 與 Java 的常見實作屬此類。

檢查。 必須把 mutex、predicate、對應 condition queue 與實作語意分開。

第 5 章練習解答　程序同步 （續）

原創驗算程式附錄: lost_update.py

```
"""Deterministic lost-update interleaving.

Run:
    python lost_update.py
"""


def unsafe_interleaving() -> int:
    balance = 100
    # Two writers both read the old value before either writes back.
    p1_read = balance
    p2_read = balance
    balance = p1_read + 50
    balance = p2_read + 50
    return balance


def serialized_updates() -> int:
    balance = 100
    balance = balance + 50
    balance = balance + 50
    return balance


def main() -> None:
    assert unsafe_interleaving() == 150
    assert serialized_updates() == 200
    print("unsafe=150 serialized=200")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study